EXPLORER UNIVERSE STUDIO — IMPLEMENTATION TECHNICAL SPECIFICATION

EXPLORER UNIVERSE PRODUCTION SYSTEM

Technical Design Document v1.1 (Master Audited Edition)

STATUS:	VERSION:	ROLE:	TARGET:
AUDIT PASSED / LOCKED	1.1 (Master Edition)	Lead Technical Architect	AI Agents & Engineers

ARCHITECTURAL AUDIT VERIFICATION MATRIX

Prior to locking this technical design document, an internal architecture audit verified all system capabilities against the six core institutional requirements:

AUDIT CHECKPOINT	STATUS	ENGINEERING IMPLEMENTATION MECHANISM
1. Supports incomplete present docs?	PASSED	Provisional Canon Engine & Dependency Resolution Service (`eups_provisional_link`).
2. Manual creation of future elements?	PASSED	Universal Entity Builder & Dynamic Schema Registry (`eups_entity_type_registry`).
3. Living OS vs. Fixed Database?	PASSED	Event-driven Graph DAG + Dynamic Schema Form Engine + Continuous Vector Indexing.
4. Manual + AI workflow in ALL modules?	PASSED	Dual-Track Pipeline: Human Exclusive Write + AI Candidate Storage (`eups_ai_output`).
5. Canon protected from overwrite?	PASSED	PostgreSQL DB-level `BEFORE UPDATE/DELETE` Write-Lock Triggers + SHA-256 Hashes.
6. Future expansion without redesign?	PASSED	JSONB Polymorphic Schema Validation + Dynamic GraphQL/REST Endpoint Resolver.

PART 1: TECHNOLOGY ARCHITECTURE

SYSTEM LAYER	TECHNOLOGY CHOICE	ARCHITECTURAL JUSTIFICATION
Frontend	Next.js 14 App Router, TailwindCSS, Shadcn UI, Zustand, TanStack Query v5	Combines SSR for heavy authority bibles with CSR for real-time dynamic workspace inspectors and graph trees.
Backend APIs	NestJS (API Gateway), Go (Golang Validation Microservice), GraphQL, REST, gRPC	NestJS handles enterprise routing and dependency injection; Go provides low-latency graph traversals and contradiction checks.
Relational Database	PostgreSQL 16 (JSONB, PostGIS, Write-Lock DB Triggers)	ACID transactional engine enforcing hard database-level locks on locked canon and storing dynamic JSON schemas.
Graph Database	Neo4j Enterprise	Houses the Asset Dependency Matrix (ADM), spatial containment trees, and causal timeline DAGs.
Vector Database	Qdrant (Distributed Vector DB)	Specialized high-dimensional vector search for Retrieval-Augmented Generation (RAG) and lore non-contamination checks.

PART 2: APPLICATION ARCHITECTURE

The system routes client interactions through an API Gateway into domain services, backed by automated Go-based validation engines and multi-tier persistence engines.

```
+-------------------------------------------------------------------------------------+
|                                   FRONTEND LAYER                                    |
|  [Next.js App Router] --> [Dynamic UI Schema Engine] --> [Workspace Views]          |
+-----------------------------------------+-------------------------------------------+
                                          | HTTPS / WSS / GraphQL
+-----------------------------------------v-------------------------------------------+
|                              API & GATEWAY ROUTING LAYER                            |
|  [NestJS Gateway] --> [RBAC & Authority Guard] --> [Dynamic Endpoint Router]        |
+-----------------------------------------+-------------------------------------------+
                                          |
     +------------------------------------+------------------------------------+
     |                                    |                                    |
     v                                    v                                    v
+-------------------------+ +-------------------------+ +-------------------------+
|  DOMAIN SERVICES LAYER  | |     AI ENGINE LAYER      | |  VALIDATION ENGINE (Go)  |
| - Universe Core Service | | - Dynamic Schema Agent   | | - Dynamic Contradiction  |
| - Authority Engine Svc  | | - Agent Coordinator      | | - ADM Graph DAG Checker  |
| - Asset & SODI Service  | | - Candidate Generator    | | - Write-Lock DB Triggers |
| - Dynamic Entity Service | |                          | | - Non-Contamination Guard|
+------------+------------+ +------------+-------------+ +------------+-------------+
             |                           |                            |
             +---------------------------+----------------------------+
                                         |
+-----------------------------------------v-------------------------------------------+
|                                 PERSISTENCE DATA BUS                                |
|   [PostgreSQL 16]   |   [Neo4j Graph DB]   |   [Qdrant Vector]   |   [S3 Storage] |
+-------------------------------------------------------------------------------------+
```

PART 3: DATABASE DESIGN

3.1 Master Entity Type Registry & Universal Table

```
-- Entity 1: Master Entity Type Registry
CREATE TABLE eups_entity_type_registry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    type_code VARCHAR(64) UNIQUE NOT NULL, -- 'SPECIES', 'TECHNOLOGY', 'CHARACTER'
    display_name VARCHAR(255) NOT NULL,
    authority_tier_default INT CHECK (authority_tier_default BETWEEN 1 AND 10),
    json_schema JSONB NOT NULL, -- Structural attribute validation rules
    is_native BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Entity 2: Master Universal Entity Table
CREATE TABLE eups_universal_entity (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    entity_type_code VARCHAR(64) REFERENCES eups_entity_type_registry(type_code),
    entity_code VARCHAR(64) UNIQUE NOT NULL, -- e.g., 'SPEC-000045'
    title VARCHAR(255) NOT NULL,
    authority_tier INT NOT NULL CHECK (authority_tier BETWEEN 1 AND 10),
    lifecycle_state VARCHAR(32) NOT NULL, -- 'CREATION','DRAFT','REVIEW','VALIDATION','APPROVAL','CANONICAL','PR
    approval_status VARCHAR(32) NOT NULL, -- 'DRAFT','PENDING_REVIEW','PROVISIONAL_CANON','APPROVED','LOCKED'
    dynamic_attributes JSONB NOT NULL,
    current_version INT DEFAULT 1,
    is_locked BOOLEAN DEFAULT FALSE,
    sha256_hash VARCHAR(64),
    created_by_user_id UUID NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

3.2 Inviolable DB Write-Protect Trigger (SQL)

```
CREATE OR REPLACE FUNCTION enforce_canon_lock_protection()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.is_locked = TRUE THEN
        RAISE EXCEPTION 'CRITICAL CANONICAL ERROR: Record [%] is LOCKED_CANON. Direct mutation or deletion is pr
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_protect_locked_canon
BEFORE UPDATE OR DELETE ON eups_universal_entity
FOR EACH ROW
EXECUTE FUNCTION enforce_canon_lock_protection();
```

PART 4: MODULE CAPABILITY MATRIX (MANUAL + AI)

MODULE NAME	MANUAL CAPABILITIES (HUMAN EXCLUSIVE)	AI CAPABILITIES (CANDIDATE & ANALYSIS)
1. Universe Core	Set physical constants, lock system laws, issue overrides.	Rule collision impact analysis, parameter cascade modeling.
2. World Management	Define biomes, edit geography, set environmental parameters.	Ecological plausibility simulations, climate route modeling.
3. Civilization Auth.	Author Guild codes (`SB-001`), set institutional rules.	Faction behavioral compliance audit, socio-economic checks.
4. Character Auth.	Create dossiers, map lineages, edit guild skill/rank progress.	Voice/dialogue consistency checks, timeline paradox scans.
5. Location Auth.	Design spatial layouts, set lighting & acoustic footprints.	Acoustic reverberation calculations, visual bible checks.
6. Asset Authority	Register SODI items, map ADM dependencies, write AER specs.	Modular family optimization, 3D geometry defect scans.
7. Story Authority	Script logs, set narrative beats, assign visitor framing layers.	Colloquialism/trope scan (`SB-004`), auto-tagging IDs.
8. Production Mgmt.	Perform gate sign-offs, issue work orders, advance lifecycle.	Production readiness scoring, ADM bottleneck prediction.
9. Timeline Mgmt.	Map temporal markers, establish causal dependency chains.	Historical paradox detection, travel time calculation.
10. Canon Mgmt.	Issue SHA-256 locks, restore old versions, merge branches.	Repository-wide canon drift scan, delta summaries.

PART 5: CONTROLLED LIFECYCLE ENGINE

Every created element (native or dynamic) must strictly traverse the unified **7-Stage Lifecycle Machine**:

```
[1. CREATION] --> [2. DRAFT] --> [3. REVIEW] --> [4. VALIDATION] --> [5. APPROVAL]
                                                                          |
[7. PRODUCTION USAGE] <--------------------- [6. CANONICAL RECORD] <------+
```

LIFECYCLE STAGE	TARGET STATE ENUM	REQUIRED ROLE	AUTOMATED GATE TRIGGER
1. Creation	`CREATION`	Creator, Writer	Schema validation against Entity Registry.
2. Draft	`DRAFT`	Creator, Writer	Complete required attribute check.
3. Review	`REVIEW`	Reviewer	Pre-review Non-Contamination Scan.
4. Validation	`VALIDATION`	Canon Guardian AI	Full Contradiction & ADM Graph Scan.
5. Approval	`APPROVAL`	Approver (Tier 1-3)	Human peer sign-off & score approval.
6. Canonical	`CANONICAL`	Approver, Admin	Cryptographic SHA-256 Hash Lock.
7. Production	`PRODUCTION`	Production Director	ADM completeness & build readiness.

PART 6: SPECIALIZED AI SUBSYSTEMS

AI SUBSYSTEM	FUNCTIONAL SCOPE	TARGET VECTOR / DATA KNOWLEDGE SOURCE	SYSTEM AUTHORITY SCOPE
Universe Assistant AI	System interface routing & query assistance.	API Schemas & Studio System Documents	Read-Only across all modules.
Canon Guardian AI	Background contradiction & non-contamination checks.	Qdrant Vector Store (`Books I-IV`, `SB-001`..`004`)	System Auditor (Writes to Diagnostic Logs).
Research AI	Deep archival searching & material property lookup.	PostgreSQL + Neo4j + Qdrant Vector Store	Read-Only with cryptographic citation links.
Creative Assistant AI	Candidate drafting for text, concepts, dialogue.	Context-Injected RAG Vector Store	Draft Writer (Writes strictly to `eups_ai_output`).
Production Assistant AI	Asset dependency checks & 3D mesh analysis.	`DC-003 Production Bible` & ADM Graph DB	Production Auditor (Read-Only).

PART 7: API DESIGN & KEY ENDPOINTS

```
-- 1. Register Dynamic Entity Category
POST /api/v1/registry/entity-type
Payload: {
  "type_code": "SPECIES",
  "display_name": "Species",
  "authority_tier_default": 5,
  "json_schema": { ... }
}
Response: { "type_id": "UUID", "status": "REGISTERED" }

-- 2. Create Universal Universe Element
POST /api/v1/entities/create
Payload: {
  "entity_type_code": "SPECIES",
  "entity_code": "SPEC-000001",
  "title": "Archival Moss",
  "authority_tier": 5,
  "dynamic_attributes": { ... }
}
Response: { "entity_id": "UUID", "lifecycle_state": "CREATION", "current_version": 1 }

-- 3. Lock Canon Record
POST /api/v1/authority/lock
Payload: { "entity_id": "UUID", "authority_tier": 2 }
Response: { "status": "LOCKED_CANON", "sha256_hash": "a8f92e..." }

-- 4. Point-In-Time Historical Restore
POST /api/v1/entities/{id}/restore
Payload: { "target_version_number": 2, "restoration_reason": "Reverting unapproved modifications" }
Response: { "entity_id": "UUID", "new_version_number": 5, "status": "RESTORED_SUCCESSFULLY" }
```

PART 8: CODING AGENT EXECUTION DIRECTIVES

MANDATORY DIRECTIVES FOR AI CODING ENGINES (MONKEYCODE / CURSOR / DEVIN)

1. **PRESERVE Dynamic Extensibility:** Never create hardcoded, isolated SQL tables for future universe categories. Route domain data through `eups_entity_type_registry` and `eups_universal_entity`.
2. **NEVER Bypass DB Write-Lock Triggers:** Do not write code that disables or circumvents `trg_protect_locked_canon`. Locked canon records must remain strictly read-only.
3. **NEVER Bypass the 7-Stage Lifecycle:** Every element created MUST strictly traverse all 7 lifecycle states.
4. **NEVER Mutate Historical Version Records:** Restores MUST ALWAYS write a NEW version entry to `eups_version_history` incrementing the version counter. Never run destructive SQL `DELETE` or `UPDATE` queries on version history.
5. **ISOLATE AI Operations:** AI agents MUST NEVER write directly to production tables with `approval_status = 'APPROVED'` or `'LOCKED'`. All AI outputs MUST be written to `eups_ai_output`.

TECHNICAL ARCHITECT:
Lead Systems Engineer
Explorer Universe Studio

Digital Signature Verified

SPECIFICATION STATUS:
Audit Passed / Full Lock
Document Code: `EUPS-TDD-V1.1`

APPROVED FOR CODING AGENT IMPLEMENTATION